에소테릭 언어 모델

Subham Sekhar Sahoo*[1] Zhihan Yang*[2] Yash Akhauri†[1] Johnna Liu†[1] Deepansha Singh†[1] Zhoujun Cheng†[3] Zhengzhong Liu[3] Eric Xing[3] John Thickstun[2] Arash Vahdat[4] *공동 제1 저자 †공동 제2 저자 [1] 코넬 테크 [2] 코넬 대학교 [3]MBZUAI [4] 엔비디아 {ssahoo, zhihany}@cs.cornell.edu

초록

확산 기반 언어 모델은 병렬 및 제어 가능한 생성을 가능하게 함으로써 자기회귀(AR) 모델에 대한 매력적인 대안을 제공합니다. 이 모델군 중에서 마스크 확산 모델(MDM)은 가장 강력한 성능을 달성하지만, 퍼플렉서티 면에서는 여전히 AR 모델에 미치지 못하며, 핵심적인 추론 시간 효율성 기능, 특히 KV 캐싱이 부족합니다. 본 연구에서는 AR 및 MDM 패러다임을 융합하여 각 패러다임의 퍼플렉서티 간에 부드러운 보간을 가능하게 하고 각자의 한계를 극복하는 새로운 모델군인 Eso-LM을 소개합니다. Eso-LM은 표준 언어 모델링 벤치마크에서 새로운 최첨단 성능을 확립합니다. 결정적으로, 우리는 병렬 생성을 유지하면서 MDM에 KV 캐싱을 도입한 최초의 연구이며, 이는 추론 효율성을 크게 향상시킵니다. 최적화된 샘플링 스케줄과 결합된 우리의 방법은 표준 MDM보다 최대 65배, 이전의 준자기회귀(semi-autoregressive) 접근 방식보다 4배 빠른 추론을 달성합니다. 프로젝트 페이지에서 코드와 모델 체크포인트를 제공합니다.

https://s-sahoo.com/Eso-LMs

1 서론

우리는 언어 모델링에 있어 중대한 순간에 있습니다. 패러다임 전환이 진행 중입니다. 오랫동안 표준으로 여겨져 온 자기회귀(AR) 언어 모델은 이제 표준 언어 생성을 위한 확산 언어 모델과 경쟁하고 있습니다. 최근 연구 *[26, 29]*에 따르면 마스크 확산 모델(MDM)은 소규모 언어 벤치마크에서 AR 모델과의 격차를 좁히고 있으며, 분자 생성 [28, 11] 및 그래프 생성 *

이러한 결과는 MDM을 AR 모델에 대한 매력적인 대안으로 만듭니다. 그러나 MDM은 두 가지 주요 한계점을 가지고 있습니다. (1) 추론 속도: 병렬 생성을 지원함에도 불구하고, MDM은 실제로는 AR 모델보다 상당히 느립니다. 이는 주로 채팅 시스템과 같은 실시간 애플리케이션에 필수적인 최적화 기술인 KV 캐싱이 부족하기 때문입니다. (2) 생성 품질: MDM은 더 복잡한 언어 모델링 작업에서 여전히 현저한 가능성(likelihood) 격차를 보입니다 *

최근 제안된 BD3-LM *

!

캐싱: 블록 간 KV 캐싱은 가능하지만, 블록 내 확산은 여전히 KV 지원이 부족하여 전반적인 속도 향상을 제한합니다.

이러한 문제들을 해결하기 위해 우리는 자기회귀 및 마스크 확산 접근 방식을 융합하는 새로운 언어 모델링 패러다임을 제안합니다. 우리의 모델은 AR 및 MDM 목표의 조합인 하이브리드 손실(hybrid loss)로 학습되며, 이는 퍼플렉서티 측면에서 두 패러다임 사이를 부드럽게 보간할 수 있도록 합니다. 이를 위해서는 두 가지 핵심 혁신이 필요합니다. (1) AR 및 MDM 방식의 생성을 모두 지원하기 위한 노이즈 제거 트랜스포머 내 수정된 어텐션 메커니즘. (2) 확산 단계 내에서 KV 캐싱을 가능하게 하는 새로운 훈련 및 샘플링 절차. 이는 이전에 MDM에서는 사용할 수 없었던 기능입니다. 이러한 하이브리드 설계의 비전통적인 특성으로 인해, 우리는 우리의 방법을 에소테릭 언어 모델(Esoteric Language Models, Eso-LMs)이라고 명명합니다.

우리의 방법은 언어 모델링 벤치마크에서 확산 모델 중 새로운 최첨단 성능을 확립하며, AR 및 MDM 퍼플렉서티 간 보간에 대한 미세 제어를 가능하게 합니다. 이전 접근 방식과 달리, 우리의 방법은 확산 중 KV 캐싱을 지원하여 추론 속도

를 현저히 향상시킵니다. 요약하자면, 우리의 기여는 다음과 같습니다. (1) 우리는 AR 및 MDM 패러다임을 융합하고 이전의 하이브리드 접근 방식인 BD3-LM보다 우수한 새로운 언어 모델링 프레임워크를 제안합니다. (2) 우리는 병렬 생성을 유지하면서 MDM에 KV 캐싱을 가능하게 한 최초의 접근 방식이며, 표준 MDM보다 최대 65×배, KV 캐싱을 적용한 준자기회귀(semi-autoregressive) 기준선보다 3−4×배 빠른 추론을 달성합니다(섹션 5.2). (3) 우리는 BD3-LM이 낮은 샘플링 단계에서 성능이 저하되는 반면, 우리의 방법은 낮은 NFE(Number of Function Evaluations) 영역에서 MDM과, 높은 NFE 영역에서 AR과 경쟁력을 유지함을 보여줍니다(섹션 5.3). (4) 우리의 결과는 이산 확산(discrete diffusion)에 대한 새로운 최첨단 퍼플렉서티를 확립하여 자기회귀 모델과의 격차를 좁힙니다(섹션 5.1).

2 배경

표기법 우리는 K개의 값을 가질 수 있는 스칼라 이산 확률 변수를 '원-핫(one-hot)' 열 벡터로 나타내며, $\mathcal{V}\in\{\mathbf{x}\in\{0,1\}^K:\Sigma_{i=1}^K\mathbf{x}^i=1\}$을 이러한 모든 벡터의 집합으로 정의합니다. $\mathbf{m}\in\mathcal{V}$를 $\mathbf{m}^K=1$을 만족하는 특수한 마스크 벡터라고 합시다. $\mathrm{Cat}(\cdot;\pi)$를 K개의 클래스에 대한 범주형 분포로 정의하며, 확률은 $\pi\in\Delta^K$으로 주어지고, 여기서 Δ^K은 K-심플렉스를 나타냅니다. 또한, $\langle\mathbf{a},\mathbf{b}\rangle$를 벡터 $\mathbf{a}$과 $\mathbf{b}$ 사이의 내적(dot product)으로 나타냅시다. 우리는 순서가 있는 집합(튜플)을 나타내기 위해 괄호 ()를 사용하고, 순서가 없는 집합을 나타내기 위해 중괄호 {}를 사용합니다. $|A|$은 집합 A의 카디널리티(원소의 개수)를 나타냅니다.

MDM은 두 가지 주요 순서(orderings)를 특징으로 합니다: 시퀀스 순서와 노이즈 제거 순서. 우리는 이 순서들 간의 관계를 설명하기 위해 순열 σ을 사용합니다. $\mathcal{P}^L$을 $[L]=\{1,\cdots,L\}$의 모든 순열 집합이라고 합시다. $\sigma\in\mathcal{P}^L$은 순서가 있는 집합(튜플)이며 전단사 함수(bijective function) 역할도 합니다. $\sigma(l)$는 σ의 lth 위치에 있는 항목이고, $\sigma^{-1}(i)$은 σ에서 항목 i의 위치입니다. 예를 들어, (2,4,1,3)은 {1,2,3,4}의 순서를 나타내며, $\sigma(2)=4$는 위치 2를 해당 항목 4에 매핑하고, $\sigma^{-1}(4)=2$은 항목 4를 해당 위치 2에 매핑합니다.

$\mathbf{x}\in\mathcal{V}^L$를 마스크 토큰이 없는 길이 L의 시퀀스라고 하고, $\mathbf{x}^\ell$를 $\mathbf{x}$의 ℓth번째 항목이라고 합시다. 우리는 '토큰 인덱스'라는 용어를 원래 순서에서 토큰의 위치를 지칭하는 데 사용합니다. 예를 들어, $\mathbf{x}^l$의 토큰 인덱스는 l입니다. $(\mathbf{z}_t)_{t\in[0,1]}\in\mathcal{V}^L$을 마스크 토큰을 포함할 수 있는 길이 L의 시퀀스라고 합시다. $\mathcal{M}(\mathbf{z}_t)=\{\ell|\mathbf{z}_t^\ell=\mathbf{m}\}$를 $\mathbf{z}_t$ 내 마스크 토큰의 토큰 인덱스라고 하고, $\mathcal{C}(\mathbf{z}_t)=\{\ell|\mathbf{z}_t^\ell\neq\mathbf{m}\}$을 $\mathbf{z}_t$ 내 깨끗한(clean) 토큰의 토큰 인덱스라고 합시다.

$\oplus:\mathcal{V}^m\times\mathcal{V}^n\to\mathcal{V}^{m+n}$를 길이 m과 n인 두 시퀀스 $\mathbf{x}=(\mathbf{x}^1,\mathbf{x}^2,\cdots,\mathbf{x}^m)$과 $\mathbf{z}=(\mathbf{z}^1,\mathbf{z}^2,\cdots,\mathbf{z}^n)$에 대한 연결 연산자(concatenation operator)라고 합시다. 연결된 시퀀스 $\mathbf{x}\oplus\mathbf{z}$가 트랜스포머에 입력될 때, $\mathbf{x}$과 $\mathbf{z}$은 독립적으로 트랜스포머에 입력될 때와 동일한 위치 임베딩을 가집니다. $\odot:\mathcal{V}^m\times\mathcal{V}^n\to\mathcal{V}^m$을 치환 연산자(substitution operator)라고 합시다. $\mathbf{z}\in\mathcal{V}^m$와 $\mathbf{x}\in\mathcal{V}^n$이 $m>n$을 만족할 때, 출력 $\mathbf{y}=\mathbf{z}\odot\mathbf{x}$는 $\mathbf{y}^{1:n}=\mathbf{x}$과 $\mathbf{y}^{n+1:m}=\mathbf{z}^{n+1:m}$로 주어집니다.

2.1 자기회귀 모델

$\mathcal{V}^L$에 있는 시퀀스 $\mathbf{x}\sim q_{\text{data}}$가 주어졌을 때, 자기회귀(AR) 모델은 결합 분포의 다음 인수분해를 정의합니다: $\log p_\theta(\mathbf{x})=\Sigma_{t=1}^L\log p_\theta(\mathbf{x}^\ell|\mathbf{x}^{c_\ell})$. 여기서 모델 p_θ은 일반적으로 인과적 트랜스포머 *

2.2 마스크 확산 모델

확산 모델은 순방향 손상 과정 q을 역전시키는 방법을 학습합니다. 이 과정은 $\mathcal{V}^L$의 깨끗한 데이터 $\mathbf{x}\sim q_{\text{data}}$을 $t\in[0,1]$에 대한 일련의 잠재 변수 $\mathbf{z}_t$으로 변환하며, 각 잠재 변수는 $\mathbf{x}$의 점점 더 노이즈가 많은 버전을 나타냅니다 [31, 32, 8]. MDM *[26, 30, 20]*에서 순방향 마스킹 과정 q은 시퀀스 $\mathbf{x}$에 걸쳐 독립적으로 인수분해됩니다. 즉, $q_t(.|\mathbf{x})=\Pi_\ell q_t(.|\mathbf{x}^\ell)$이며 각 토큰 $\mathbf{x}^l$는 고정된 목표 분포 $\mathrm{Cat}(.;\mathbf{m})$와 점진적으로 보간됩니다. 시간 t에서의 $\mathbf{z}_t^\ell\sim q_t(.|\mathbf{x}^\ell)$의 한계분포(marginal)는 다음과 같습니다:

$$q_t(.|\mathbf{x}^\ell)=\mathrm{Cat}(.;\alpha_t\mathbf{x}^\ell+(1-\alpha_t)\mathbf{m}),\quad(1)$$

여기서 αt∈[0,1]는 t에 대해 엄격하게 감소하는 함수이며, α0≈1와 α1≈0을 만족합니다. *Sahoo et al.

[$q_{s|t}(.|₩mathbf{z}_t^{₩ell}, ₩mathbf{x}^{₩ell}) = ₩begin{cases} ₩text{Cat}(.; ₩mathbf{z}_t^{₩ell}) & ₩mathbf{z}_t^{₩ell} ₩neq ₩mathbf{m}, ₩ ₩text{Cat}₩left(:{₩frac{(1-₩alpha_s)₩mathbf{m}+({₩alpha_s-₩alpha_t})₩mathbf{x}^{₩ell}}{1-₩alpha_t}}₩right) & ₩mathbf{z}_t^{₩ell} = ₩mathbf{m}. ₩end{cases}]$ (2)

노이즈 제거 모델 $\mathbf{x}\theta:\mathcal{V}^L \to (\Delta^K)^L$이 주어졌을 때, 시퀀스 $\mathbf{z}_s$에 대한 역 언마스킹(unmasking) 과정 $p\theta(.|\mathbf{z}t)s|t$은 다음과 같이 매개변수화됩니다.

$p\theta(.|\mathbf{z}t)s|t = \Pi_\ell L p\theta_\ell(.|\mathbf{z}t)s|t = \Pi_t L q s|t_\ell(.|\mathbf{z}t_\ell, \mathbf{x}_\ell = \mathbf{x}\theta_\ell(\mathbf{z}t))$. (3)

*Sahoo et al.

$\mathscr{L}MDM(\mathbf{x}) = \mathbb{E}q,t\sim[0,1]\alpha t' 1-\alpha t \Sigma_{\ell \in \mathscr{M}(\mathbf{z}t)} \log\langle \mathbf{x}\theta_\ell(\mathbf{z}t), \mathbf{x}_\ell \rangle$, (4)

이는 마스크된 토큰 $\mathscr{M}(\mathbf{z}t)$에 대해서만 계산된 마스크 언어 모델링 손실 *

길이 L의 시퀀스를 생성하기 위해 역확산 프로세스는 $\mathbf{z}t=1_\ell = \mathbf{m}$는 $\ell=1,\cdots,L$입니다. 이 프로세스는 T 단계에 걸쳐 진행되며, 각 $\mathbf{z}s_\ell$는 (3)에서 정의된 대로 $p\theta(.|\mathbf{z}t)s|t$에서 독립적으로 샘플링됩니다. 일단 위치가 언마스킹되면 고정됩니다. 여러 토큰을 병렬로 노이즈 제거할 수 있으므로 총 네트워크 함수 평가(NFE) 수는 L보다 작을 수 있어 더 빠른 생성이 가능합니다. 그러나 $\mathbf{x}\theta(\mathbf{z}t)$의 양방향 트랜스포머로 인해 각 NFE는 비용이 많이 듭니다.

2.3 블록 노이즈 제거 확산 이산 언어 모델

블록 노이즈 제거 확산 이산 언어 모델(BD3-LM) *

블록의 크기를 변경함으로써 BD3-LM은 AR 모델과 MDM을 보간합니다. BD3-LM은 $\mathbf{x}$의 토큰을 B개의 L' 연속 토큰 블록으로 그룹화하며, 여기서 $B=L/L'$이고 B는 정수라고 가정합니다. $\mathbf{x}$에 대한 우도(likelihood)는 블록에 대해 자기회귀적으로 $-\log p\theta(\mathbf{x}) = -\Sigma_{b=1}^{B} \log p\theta(\mathbf{x}b|\mathbf{x}<b) \leq \Sigma_{b=1}^{B} \mathscr{L}MDM(\mathbf{x}b;\mathbf{x}<b)$으로 인수분해되며, 여기서 $p\theta(\mathbf{x}b|\mathbf{x}<b)$은 조건부 MDM이고 $\mathscr{L}MDM(\mathbf{x}b;\mathbf{x}<b)$은 (4)에서 정의된 MDLM에 대한 NELBO로, 블록 전체에 순차적으로 적용됩니다.

3 난해한 언어 모델

우리는 언어 모델링을 위한 새로운 패러다임인 난해한 언어 모델(Esoteric Language Models, Eso-LMs)을 제안합니다. 이는 AR 모델과 MDM의 공생적 결합을 이룹니다. 우리의 접근 방식은 두 가지 주요 구성 요소를 가집니다: (1) 우리는 순차적(AR 방식) 및 병렬(MDM 방식) 생성을 모두 지원하는 모델군을 도입합니다; 그리고 (2) 섹션 4에서 이 하이브리드 생성을 가능하게 하는 트랜스포머 *

AR 모델은 현재 언어 모델링 성능에서 앞서고 있지만, 토큰을 순차적으로 생성하여 추론 속도가 느립니다. 이와 대조적으로 MDM은 여러 토큰을 병렬로 생성하며 더 제어 가능하지만 [29, 19], 일반적으로 AR 모델보다 더 높은 퍼플렉시티를 산출합니다 [26, 27]. 핵심 질문은 다음과 같습니다: 이들의 강점을 결합할 수 있을까요? 우리는 MDM을 통해 일부 토큰을 병렬로 생성하고 나머지는 좌에서 우로 순차적으로 생성하는 하이브리드 접근 방식을 도입합니다. 이는 두 가지 주요 질문을 제기합니다. 첫째, 이러한 생성 모델의 우도를 계산할 수 있을까요? 우리는 Eso-LM이 실제 우도에 대한 원칙적인 경계를 허용함을 보여줍니다. 둘째, 트랜스포머의 어텐션 메커니즘을 두 가지 생성 스타일을 모두 지원하도록 어떻게 조정할 수 있을까요? 훈련 중에 AR 모델에서는 깨끗한 시퀀스에 인과적 어텐션이 강제되는 반면, MDM은 마스킹된 토큰과 마스킹되지 않은 토큰 모두에 대한 양방향 어텐션에 의존합니다. 섹션 4에서는 트랜스포머를 위한 맞춤형 어텐션 메커니즘을 설계하여 이 두 가지 별개의 아키텍처를 조화시킵니다. 또한, 어텐션 행렬에 희소성을 도입하여 MDM에 대한 KV-캐싱을 가능하게 함으로써 긴 컨텍스트 생성 속도를 크게 향상시킵니다.

3.1 자기회귀 모델과 마스크 확산 융합

데이터 분포에서 추출된 샘플을 $\mathbf{x}$~qdata($\mathbf{x}$) in $\mathcal{V}$L라고 하고, θ으로 파라미터화된 모델 분포를 pθ라고 합시다. Eso-LMs는 pθ를 두 가지 구성 요소로 분해합니다: AR 모델 pθAR와 MDM pθMDM. MDM은 부분적으로 마스킹된 시퀀스 $\mathbf{z}0\in\mathcal{V}$L~pθMDM($\mathbf{z}$0)을 생성하고, AR 모델은 나머지 언마스킹 단계를 자기회귀적인 좌-우 방식으로 완료합니다: pθAR($\mathbf{x}|\mathbf{z}$0). 이러한 하이브리드 생성 프로세스의 한계 우도(marginal likelihood)는 다음과 같습니다:

pθ($\mathbf{x}$)=Σ$\mathbf{z}0\in\mathcal{V}$LpθAR($\mathbf{x}|\mathbf{z}$0)pθMDM($\mathbf{z}$0). (5)

이 합은 다루기 어렵지만, 사후 분포 q($\mathbf{z}$0|$\mathbf{x}$) *

−logpθ($\mathbf{x}$) ≤−$\mathbb{E}\mathbf{z}$0~q0(.|$\mathbf{x}$)[logpθAR($\mathbf{x}|\mathbf{z}$0)]+DKL(q0($\mathbf{z}$0|$\mathbf{x}$) ‖ pθMDM($\mathbf{z}$0)) (6) =−$\mathbb{E}\mathbf{z}$0~q0(.|$\mathbf{x}$) [Σ$\ell\in\mathcal{M}$($\mathbf{z}$0)logpθAR($\mathbf{x}\ell|\mathbf{z}$0,$\mathbf{x}<\ell$)]+DKL(q0($\mathbf{z}$0|$\mathbf{x}$) ‖ pθMDM($\mathbf{z}$0)).

여기서 pθAR($\mathbf{x}|\mathbf{z}$0)은 마스킹된 위치 $\ell\in\mathcal{M}$($\mathbf{z}$0)에 대한 결합 AR 우도이며, $\mathbf{z}$0의 깨끗한 토큰에 조건화됩니다. AR의 경우, 노이즈 제거 네트워크 $\mathbf{x}$θ:$\mathcal{V}$L→(ΔK)L은 입력 $\mathbf{z}0\odot\mathbf{x}<\ell$에서 작동하며, 여기서 치환 연산자 ⊙는 $\mathbf{z}$0의 처음 l−1개 토큰을 $\mathbf{x}<\ell$로 대체합니다. 각 $\ell\in\mathcal{M}$($\mathbf{z}$0)에 대해 $\mathbf{x}\theta\ell$($\mathbf{z}0\odot\mathbf{x}<\ell$)은 $\mathbf{x}<\ell$ 및 $\mathbf{z}$0이 주어졌을 때 깨끗한 토큰 $\mathbf{x}\ell$의 분포를 근사하며, 이는 위치 ℓ를 넘어선 깨끗한 토큰을 포함할 수 있습니다. 보충 자료 B.1에서 우리는 KL 항을 분석하고 NELBO가 다음임을 보여줍니다.

$\mathscr{L}$NELBO($\mathbf{x}$)=$\mathbb{E}\mathbf{z}$0~q0[−Σ$\ell\in\mathcal{M}$($\mathbf{z}$0)log⟨$\mathbf{x}\theta\ell$($\mathbf{z}0\odot\mathbf{x}<\ell$),$\mathbf{x}\ell$⟩]⏟AR loss+∫t=0t=1αt′1−αt$\mathbb{E}\mathbf{z}$t~qt[Σ$\ell\in\mathcal{M}$($\mathbf{z}$t)log⟨$\mathbf{x}\theta\ell$($\mathbf{z}$t),$\mathbf{x}\ell$⟩]⏟M (7)

AR과 확산 사이의 보간

α0=1일 때, 사후 샘플은 $\mathbf{z}$0=$\mathbf{x}$이고 모든 토큰은 MDM에 의해 생성됩니다. 따라서 (7)에서 AR 손실은 0이 되고, $\mathscr{L}$NELBO는 MDM 손실로 감소합니다. 반대로, α0=0일 때 모든 토큰이 마스킹되고 MDM 손실은 사라져 $\mathscr{L}$NELBO는 AR 손실로 감소합니다. 따라서 Eso-LM은 하이퍼파라미터 α0에 의해 제어되는 AR 및 MDM 패러다임 사이를 보간합니다.

3.2 샘플링

우리는 (5)에 정의된 2단계 샘플링을 수행합니다: pθ($\mathbf{x}$)에서 샘플링하기 위해, 먼저 부분적으로 마스킹된 시퀀스 $\mathbf{z}$0~pθMDM을 그리고 나서 pθAR를 사용하여 이를 완성합니다.

확산 단계

완전히 마스킹된 시퀀스 $\mathbf{z}$t=1=$\mathbf{m}$1:L에서 시작하여, 표준 선조 샘플링(섹션 2.2)은 각 단계에서 마스킹된 토큰의 일부만 업데이트하면서 전체 시퀀스에 대한 정방향 패스를 수행하여 FLOP을 낭비합니다. 효율성을 개선하기 위해, 우리는 표준 샘플링 및 훈련 절차에 두 가지 주요 변경 사항을 제안합니다. 샘플링 중에 우리는 확산 노이즈 제거 스케줄 $\mathcal{S}$MDM=(S1,⋯,S1/T)을 미리 계산하는데, 여기서 St는 확산 단계 t에서 노이즈 제거될 마스크 토큰 인덱스 집합입니다. 전체 시퀀스를 처리하는 대신, 우리는 부분 시퀀스 {$\mathbf{z}$t$\ell|\ell\in\mathcal{C}$($\mathbf{z}$t)∪St} (즉, 깨끗한 토큰과 노이즈 제거 대상으로 예정된 토큰)에 대해서만 정방향 패스를 수행하여, 특히 긴 시퀀스에서 계산을 상당히 줄입니다. 이러한 샘플링 방식을 맞추기 위해, 모델은 훈련 중에 마스크 토큰에 대한 양방향 어텐션을 피해야 합니다(섹션 4에서 논의). 이 접근 방식은 또한 확산 중 효율적인 KV-캐싱을 가능하게 합니다(섹션 4.2에서 논의).

순차적 단계

최종 출력 $\mathbf{x}$~을 $\mathbf{z}0$으로 초기화합니다. AR 모델은 마스크 토큰을 좌-우 순서로 $\mathcal{S}$AR=((i)|i∈$\mathcal{M}$($\mathbf{z}0$)) 노이즈 제거 스케줄에 따라 채웁니다. 이를 위해 단일 원소 집합은 유일한 원소를 기준으로 오름차순으로 정렬되어야 합니다. 표준 AR 디코딩과 달리, 각 $\mathbf{x}$~ℓ은 깨끗한 토큰으로만 구성된 좌측 컨텍스트와 $\mathbf{z}0$의 우측 깨끗한 토큰 모두에 조건화되어 더 풍부한 생성을 가능하게 합니다. 우리는 우측 마스크 토큰에 대한 평가를 건너뛰어 불필요한 계산을 줄입니다. 순차적 단계는 자연스럽게 KV 캐싱을 지원합니다.

우리는 두 샘플링 스케줄을 연결하여 [L]을 분할하는 통합 노이즈 제거 스케줄을 $\mathcal{S}$=$\mathcal{S}$MDM∪$\mathcal{S}$AR으로 표기합니다. α0=1일 때, 모든 토큰은 확산에 의해 생성되므로 $\mathcal{S}$=$\mathcal{S}$MDM 및 $\mathcal{S}$AR=∅ (널 집합)입니다. α0=0일 때, 모든 토큰은 순차적으로 생성되므로 $\mathcal{S}$=$\mathcal{S}$AR 및 $\mathcal{S}$MDM=∅입니다. 전체 샘플링 알고리즘 및 예시는 보충 자료 B.4를 참조하십시오.

4 공유 노이즈 제거 트랜스포머를 위한 어텐션 메커니즘 설계

이 섹션에서는 공유 트랜스포머 아키텍처를 사용하여 순차적(AR) 및 병렬(MDM) 생성 모드를 모두 지원하는 통합 모델링 프레임워크를 소개합니다. 우리의 핵심 기술 기여는 인과적 어텐션과 1단계 시프트 디코딩을 요구하는 AR 모델과 양방향 어텐션에 의존하는 MDM 사이의 아키텍처적 불일치를 해결하는 유연한 어텐션 메커니즘입니다. 이를 달성하기 위해, 우리는 시퀀스 길이 L′인 어텐션 바이어스 행렬 A∈{−∞,0}L′×L′을 도입하여 표준 어텐션을 다음과 같이 조절합니다:

₩textscSelf−Attention(Q,K,V,A)=softmax(QK′d+A)V

여기서 Q,K,V∈ℝL′×d은 쿼리, 키, 값 행렬을 나타냅니다. A의 엔트리들은 정보 흐름을 제어합니다: Ai,j=0는 토큰 i에서 j로의 어텐션을 "허용"하고 Ai,j=−∞은 "차단"합니다. 이 메커니즘은 단일 트랜스포머가 필요에 따라 인과적 및 양방향 동작을 모두 모방할 수 있게 합니다.

이 통합 어텐션 스키마를 바탕으로, 우리는 Eso-LM (A)와 Eso-LM (B)의 두 가지 변형을 제안합니다. Eso-LM (A)는 어텐션을 희소화하고 각 확산 단계에서 노이즈 제거 트랜스포머를 마스크 토큰의 부분 집합에 선택적으로 적용함으로써 계산량 감소에 중점을 둡니다. Eso-LM (B)는 마스크 토큰뿐만 아니라 깨끗한 토큰에도 인과적 마스킹을 도입하여 이 아이디어를 더욱 확장하여, 약간의 퍼플렉시티 저하를 대가로 훨씬 더 큰 속도 향상을 위한 KV-캐싱을 가능하게 합니다. 이 섹션의 나머지 부분에서는 이러한 통합을 가능하게 하는 어텐션 공식을 설명하고, 확산 및 순차 단계 모두에서 이 두 변형 간의 차이점을 자세히 설명합니다.

확산 단계

표준 MDM은 샘플링 중에 두 가지 비효율성으로 어려움을 겪습니다: (i) 부분 업데이트에도 불구하고 전체 시퀀스 정방향 패스, (ii) 양방향으로 인해 단계 간 KV 캐싱 없음

그림 2: 확산 단계 학습을 위한 어텐션 편향 비교. 주황색은 0(어텐션)을, 회색은 −∞(어텐션 없음)을 나타냅니다. 클린 시퀀스는 $\mathbf{x}$=(A,B,C,D,E,F)입니다. 무작위 마스킹 후 $\mathbf{z}$t=(A,M,C,M,M,F)를 얻습니다. 정수는 $\mathcal{M}$($\mathbf{z}$t)={2,4,5} 및 $\mathcal{C}$($\mathbf{z}$t)={1,3,6}와 함께 위치 인덱스를 나타냅니다. 무작위 순서는 마스크 토큰 앞에 클린 토큰이 오는 σ=(3,1,6,4,5,2)~$\mathcal{P}$6입니다. 빨간색은 Eso-LM (A)와 Eso-LM (B)의 차이점을 강조합니다.

어텐션. 3.2절에서 간략히 논의했듯이, (i)은 t 단계에서 {$\mathbf{z}$ti|i∈$\mathcal{C}$($\mathbf{z}$t)∪St}에 대해서만 $\mathbf{x}$R를 평가하여 해결합니다. 이 가능성을 열기 위해, 우리는 AO-ARM

순차 단계 순차 생성 중에 모델은 $\mathbf{z}$0~pθMDM(.)에 의해 생성된 마스크 토큰 ($\mathbf{z}$0i)t∈$\mathcal{M}$($\mathbf{z}$0)를 왼쪽에서 오른쪽으로 노이즈 제거해야 합니다. 이는 생성 중에는 간단하지만 학습 중에는 어렵습니다. 각 토큰이 왼쪽만 참조하는 표준 AR 모델과 달리, 각 마스크 토큰 $\mathbf{z}$0i는 (i) 자기 자신(위치 임베딩용), (ii) 클린 왼쪽 컨텍스트 $\mathbf{x}$<i ($\mathbf{z}$0<i의 모든 이전 마스크

토큰이 채워졌다고 가정), (iii) 그리고 오른쪽 컨텍스트 {xj|j∈C(z0),j>i} 또는 동등하게 {z0j|j∈C(z0),j>i}의 클린 토큰을 참조해야 합니다. 학습 중 (ii)를 처리하는 것은 간단하지 않습니다. 왜냐하면 z0의 각 마스크 토큰은 왼쪽에 있는 토큰들이 이미 pθAR에 의해 노이즈 제거되었다고 가정하기 때문입니다. 이러한 동작을 시뮬레이션하기 위해, 우리는 학습 중에 연결된 시퀀스 z0⊕x를 트랜스포머에 입력하고 모든 마스크 토큰에 대한 올바른 정보 흐름을 강제하는 사용자 정의 어텐션 마스크 A를 구성합니다. 손실은 (7)에 따라 z0에서 마스킹된 토큰에 해당하는 로짓에 대해서만 계산됩니다. 생성 중에는 이 문제가 자연스럽게 해결되며, 연결이 필요하지 않습니다.

4.1 Eso-LM (A)

우리는 확산 단계에서 노이즈 제거 트랜스포머의 마스크 토큰에 대한 양방향 어텐션을 제거하는 Eso-LM (A)를 소개합니다. 이 수정은 3.2절에 설명된 샘플링 스케줄 사용을 가능하게 하여 더 빠른 추론을 허용합니다. Eso-LM (A)는 순차 샘플링 중에만 KV 캐싱을 지원하지만, 확산 샘플링 중에는 표준 MDM보다 훨씬 적은 계산을 수행합니다.

4.1.1 학습

확산 단계 노이즈 제거 트랜스포머는 노이즈 제거할 마스크 토큰을 포함하는 zt~qt(.|x)을 입력으로 받고, x을 목표로 받습니다. 무작위 순서 σ~P_L는 σ에서 zt의 마스크 토큰보다 zt의 클린 토큰이 선행한다는 자연스러운 제약과 함께 샘플링됩니다. 우리는 L×L 어텐션 편향을 다음과 같이 정의합니다.

Ai,j={0amp;∀(i,j)∈C(zt)×C(zt)0amp;i fσ−1(i)≥σ−1(j)∀(i,j)∈M(zt)×[L]−∞amp;o t h e r w i s e .₩tag10

클린 토큰 {zti|i∈C(zt)}는 서로 양방향 어텐션을 가지며 (8), 마스크 토큰 (zti)i∈M(zt)은 클린 토큰, 자기 자신, 그리고 σ당 이전 마스크 토큰을 참조합니다 (9). 우리는 다음을 무시한다는 점에 유의하십시오.

σ 내 클린 토큰 간의 순서를 무시하고 단순히 그들 간의 양방향 어텐션을 사용합니다. 예시는 그림 2를 참조하십시오. 흥미롭게도, A의 행과 열을 σ로 정렬하면 A은 Prefix-LM

순차 단계 노이즈 제거 트랜스포머는 연결된 시퀀스 z0⊕x∈V2L를 입력으로 받으며, 여기서 z0~q0(.|x)은 노이즈 제거할 마스크 토큰을 포함하고, x를 목표로 받습니다. 우리는 2L×2L 어텐션 편향을 다음과 같이 정의합니다.

노이즈 컨텍스트 z0 내 토큰에 대한 어텐션 메커니즘은 다음과 같이 설명됩니다. 마스크 토큰 (z0i)i∈M(z0)은 (i) 자기 자신(11), (ii) 클린 토큰 {xj|j∈C(z0)}(12), (iii) 그리고 왼쪽에 있는 마스크 토큰의 클린 버전 {xj|j∈M(z0),i>j}(13)을 참조합니다. 클린 토큰 (z0i)i∈C(z0)는 다른 어떤 토큰도 참조하지 않기 때문에 무엇이든 참조할 수 있습니다 (17). 클린 컨텍스트 x0 내 토큰에 대한 어텐션 메커니즘은 다음과 같이 설명됩니다. 토큰 {xi|i∈C(z0)}은 양방향 어텐션을 가집니다 (14). 마스크 토큰에 해당하는 클린 토큰 (xi)i∈M(z0)은 {xj|j∈C(z0)}(15) 및 {xj|j∈M(z0),i≥j}(16)를 참조합니다.

예시는 그림 6을 참조하십시오. σ는 두 가지 속성을 가진 순서라고 합시다: (i) σ에서 z0의 마스크 토큰보다 z0의 클린 토큰이 선행하고 (ii) σ에서 z0의 마스크 토큰이 자연스러운 순서로 배열됩니다. 다시 말하지만, 클린 토큰 {xi|i∈C(z0)} 간의 순서는 무시되며, 우리는 그들 간의 양방향 어텐션을 사용합니다. 각 네 개의 LxL 블록의 행과 열이 σ에 의해 정렬되면, A는 희소하고 패턴화된 블록 구조를 나타내며 (그림 8), 우리는 이 속성을 구현에 활용합니다 (보충 자료 B.5).

4.1.2 샘플링

확산 또는 순차 샘플링 중에, 부분적으로 마스킹된 시퀀스 zk가 주어졌을 때, 노이즈 제거 모델은 K=|S|인 Sk∈S={S1,⋯,SK}에 대해 마스크 토큰 {zki|i∈Sk}의 노이즈를 제거해야 합니다. 우리는 토큰의 부분 집합 {zki|i∈C(zk)∪Sk}에 대해 순방향 패스를 수행합니다. 토큰의 부분 집합에 대해 순방향 패스를 수행하는 동안 실제 시퀀스 내

이 토큰들의 위치 임베딩이 보존된다는 점에 유의하는 것이 중요합니다. 아래에서는 순방향 패스에 사용되는 어텐션 편향에 대해 논의합니다.

DkMDM를 k 단계 이전에 확산 단계에서 디코딩된 토큰 인덱스의 집합이라고 하고, DkAR를 순차 단계에 대한 것이라고 합시다. 정렬 σ을 S에 의해 정의된 토큰 노이즈 제거 순서라고 합시다. 우리는 k 단계에서 L×L 어텐션 편향을 다음과 같이 정의합니다.

확산 중 디코딩된 클린 토큰 {**z**ki|i∈DkMDM}는 서로 양방향 어텐션을 가집니다 (18). 순차적으로 디코딩된 클린 토큰 (**z**ki)i∈DkAR은 확산 중 디코딩된 클린 토큰 {**z**kj|j∈DkMDM}(19), 자기 자신, 그리고 이전 순차적으로 디코딩된 클린 토큰 {**z**kj|j∈DkAR,i>j}(20)을 참조합니다. 노이즈 제거할 마스크 토큰 (**z**ki)i∈Sk는 모든 디코딩된 클린 토큰 {**z**kj|j∈DkMDM∪DkAR}(21), 자기 자신, 그리고 σ당 이전 노이즈 제거할 마스크 토큰: {**z**kj|j∈Sk,σ−1(i)>σ−1(j)}(22)을 참조합니다. 노이즈 제거가 예정되지 않은 마스크 토큰 (**z**ki)i∈S>k은 다른 어떤 토큰도 참조하지 않기 때문에 무엇이든 참조할 수 있습니다 (23).

(i) (**z**ki)i∈$\mathcal{C}$(**z**k)∪Sk가 {**z**ki|i∈S>k}를 참조하지 않고 (ii) 클린 토큰을 샘플링하기 위해 {**z**ki|i∈Sk}에 대한 트랜스포머 출력만 사용하므로, {**z**ki|i∈S>k}에 대한 어떤 계산도 피할 수 있어 확산 단계에서 계산량이 크게 줄어듭니다. 그림 9는 이 샘플링 절차가 순차 단계 동안 KV 캐싱과 함께 예시 시퀀스를 생성하는 방법을 보여줍니다.

4.2 Eso-LM (B)

Eso-LM (A)는 순차 샘플링 중에 KV 캐싱을 수행할 수 있지만, 클린 토큰들이 서로 양방향 어텐션을 가지기 때문에 확산 샘플링 중에는 이를 수행할 수 없습니다. 따라서, 마스크 토큰 집합이 노이즈 제거된 후에는 새로 노이즈 제거된 토큰을 포함한 모든 클린 토큰에 대한 KV가 업데이트되어야 합니다. 확산 샘플링 중에 KV 캐싱을 가능하게 하기 위해, 우리는 클린 토큰들 간에 인과적 어텐션을 적용하며, 이는 Eso-LM (B)로 이어집니다. 또한 두 단계 사이에 공유 KV 캐시를 가능하게 합니다.

4.2.1 학습

확산 단계 노이즈 제거 트랜스포머는 노이즈 제거할 마스크 토큰을 포함하는 **z**t~qt(.|**x**)을 입력으로 받고, **x**을 목표로 받습니다. 우리는 MDM과 AO-ARM

Ai,j={0amp;if σ−1(i)≥σ−1(j) ∀(i,j)∈[L]×[L]−∞amp;otherwise.₩tag24

예시는 그림 2를 참조하십시오. 더욱이, A의 행과 열을 σ로 정렬하면 A는 인과적 어텐션 편향이 됩니다 (그림 7); 우리는 이 속성을 구현에 활용합니다 (보충 자료 B.5).

순차 단계 노이즈 제거 트랜스포머는 연결된 시퀀스 **z**0⊕**x**∈$\mathcal{V}$2L을 입력으로 받으며, 여기서 **z**0~q0(.|**x**)은 노이즈 제거할 마스크 토큰을 포함하고, **x**를 목표로 받습니다. 샘플링 중에는 인과적 어텐션이 적용되고 KV 캐시가 구축되는 **z**0~pϕMDM 내 클린 컨텍스트가 확산 단계에 의해 제공됩니다. 확산 단계에서 구축된 KV 캐시를 효과적으로 사용하려면, 순차 단계 학습 중에 클린 토큰 {**x**i|i∈$\mathcal{C}$(**z**0)} 간에 유사한 인과적 어텐션을 적용해야 합니다. 이를 달성하기 위해, 우리는 먼저 (i) σ에서 **z**0의 마스크 토큰보다 **z**0의 클린 토큰이 선행하고 (ii) σ에서 마스크 토큰이 자연스러운 순서로 배열된다는 제약과 함께 σ~$\mathcal{P}$L를 샘플링합니다. 그런 다음, (11)-(17)을 재사용하되, {**x**i|i∈$\mathcal{C}$(**z**0)}에 대한 양방향 어텐션 (14)을 σ당 인과적 어텐션(29)으로 대체합니다:

Ai,j=0if i=j ∀(i,j)∈$\mathcal{M}$(**z**0)×$\mathcal{M}$(**z**0)₩tag26

Ai,j+L=0∀(i,j)∈$\mathcal{M}$(**z**0)×$\mathcal{C}$(**z**0)₩tag27

Ai,j+L=0if i>j ∀(i,j)∈𝒜(z0)×𝒜(z0)₩tag28

Ai+L,j+L=0if σ−1(i)≥σ−1(j) ∀(i,j)∈𝒞(z0)×𝒞(z0)₩tag29

Ai+L,j+L=0∀(i,j)∈𝒜(z0)×𝒞(z0)₩tag30

Ai+L,j+L=0if i≥j ∀(i,j)∈𝒜(z0)×𝒜(z0)₩tag31

Ai,j=−∞otherwise.₩tag32

예시는 그림 6을 참조하라. 네 개의 L×L 블록 각각의 행과 열이 σ 기준으로 정렬되면, A는 우리가 구현에 활용하는 구조(그림 8)를 보여준다 (부록 B.5).

4.2.2 샘플링

4.1.2절의 동일한 설정 및 표기법에 따라, 우리는 (18)-(23)을 재사용하지만, DkMDM (18)에 대한 양방향 어텐션을 σ 당 인과적 어텐션으로 대체한다. 노이즈 제거 순서(33)는 다음과 같다:

Ai,j={0amp;if σ−1(i)≥σ−1(j) ∀(i,j)∈DkMDM×DkMDM0amp;∀(i,j)∈DkAR×DkMDM0amp;if i≥j ∀(i,j)∈DkAR×DkAR0am
₩tag38

두 단계 모두에서 KV 캐싱 4.1.2절에서와 같이, 우리는 미래 마스크 토큰 {zk|i∈S>k}에 대한 계산을 수행하지 않는다. 더 나아가, 이전에 디코딩된 토큰들은 확산 방식이든 순차 방식이든 상관없이 서로 인과적 어텐션을 가지므로, k 단계에서 새로 생성된 토큰들은 k+1 단계에서 KV 캐시를 구축하는 데 사용될 수 있으며, 그 캐시된 KV 값은 k+2 단계부터 사용될 수 있다. 그림 1은 이 샘플링 절차가 예시 시퀀스를 어떻게 생성하는지 보여준다.

---

5 실험

우리는 Eso-LM을 두 가지 표준 언어 모델링 벤치마크인 One Billion Words 데이터셋(LM1B) *

5.1 우도 평가

우리의 실험은 Eso-LM이 LM1B 및 OWT에서 확산 모델 중 최첨단 혼란도를 달성하며, MDM과 AR 혼란도 사이의 더 미세한 보간을 제공함을 보여준다 (표 1 및 표 2).

실험 설정 우리는 Eso-LM을 선도적인 마스크 확산 모델—MDLM *

결과 우리의 방법에서, PPL은 (7)을 사용하여 계산된다. LM1B에서, 우리는 Eso-LM (A)를 α0∈{0.0625,0.125,0.25,0.5,1.0}로, Eso-LM (B)를 α0=1로 학습한다. 우리는 Eso-LM (A)가 MDLM과 AR 혼란도 사이를 효과적으로 보간한다는 것을 발견했다. 놀랍게도, 마스크 토큰에 대한 양방향 어텐션을 인과적 마스크로 대체하여 MDM을 수정한 Eso-LM (A)는 α0=1에서 MDLM보다 ≈1 PPL 더 우수한 성능을 보인다. 이러한 개선은 앞서 설명한 저분산 학습 손실에 기인할 수 있다. 대조적으로, Eso-LM (B)는 클린 토큰들 사이에 인과적 종속성을 도입하여, α0=1에서 Eso-LM (A)보다 ≈4 PPL만큼 성능을 저하시킨다. 이는 Eso-LM (A)의 클린 토큰에 대한 양방향 어텐션이 Eso-LM (B)의 인과적 어텐션보다 엄밀히 말해 더 표현력이 높기 때문에 예상되는 결과이다. OWT에서, Eso-LM (B)는 확산 단계 동안 KV 캐싱을 지원하는 유일한 모델—BD3-LM 포함—이기 때문에 특히 매력적이다. α0∈{0.0625,0.125,0.25,0.5,1.0}으로 학습되었을 때, 이는 AR과 MDM 혼란도 사이의 부드러운 보간을 보여준다 (표 2). LM1B와 마찬가지로, Eso-LM (A)는 OWT에서도 MDLM과 경쟁력을 유지한다.

5.2 생성 속도

우리의 정성적 평가 실험은 Eso-LM이 KV 캐싱을 지원하는 이전 확산 기반 방법들보다 3–4× 더 빠르고, KV 캐싱을 지원하지 않는 MDM보다 약 최대 65× 더 빠르다는 것을 보여준다.

실험 설정 MDLM과 같은 확산 모델은 양방향 어텐션을 사용하기 때문에 KV 캐싱을 지원하지 않는다. BD3-LM과 같은 반자기회귀 모델은 토큰 블록을 자기회귀적으로 생성하고 완료된 블록을 캐싱함으로써 부분적인 KV 캐싱을 가능하게 하지만, 확산 중인 현재 블록에 대한 캐싱은 여전히 부족하다. 대조적으로, 우리의 방법은 확산 단계 동안 KV 캐싱을 독점적으로 지원한다. 우리는 우리의 방법인 Eso-LM (B)의 추론 시간을 MDLM 및 BD3-LM과 L∈{2048,8192} 컨텍스트 길이에서, first-hitting 샘플러 *

표 1: LM1B의 테스트 혼란도(PPL; ↓). *He et al.

표 2: 25만 스텝 동안 학습된 모델의 OWT 테스트 혼란도(PPL; ↓). ǂ

표 3: 모든 방법에서 NFEs를 L로 설정했을 때, 시퀀스 길이 L∈{2048,8192}에 대한 샘플링 시간(↓, 초 단위). 보고된 값은 5회 이상의 실행에 대한 평균이다.

결과 표 3에서 볼 수 있듯이, KV 캐싱이 없는 MDLM과 비교하여 Eso-LM (B)는

L=2048의 경우 ~14× 더 빠르고, L=8192의 경우 ~65× 더 빠르다. 부분적으로 캐싱을 지원하는 BD3-LM과 비교했을 때, Eso-LM (B)는 L=8192에서 BD3-LM (L′=16)보다 ~3.2× 더 빠르고 BD3-LM (L′=4)보다 ~3.8× 더 빠르다. 이러한 속도 향상은 KV 캐싱과 노이즈 제거될 마스크된 토큰 및 이전에 예측된 토큰으로 순방향 패스를 제한하여 중복 계산을 피하는 스케줄러 S에서 비롯된다—이는 BD3-LM이 부족한 기능이다. NFEs를 L로 제한하므로, 우리의 방법은 지연된 KV 재사용(마지막에서 두 번째 노이즈 제거 단계부터만 가능) 때문에 AR 모델보다 느리지만, 전체적인 효율성은 이전 확산 기반 베이스라인을 여전히 능가한다 (4.2.2절 참조).

5.3 생성 품질

우리의 샘플링 실험은 (1) Eso-LM이 샘플링 속도와 품질의 파레토 프론티어에서 새로운 SOTA를 확립하고 (그림 4), (2) 높은 NFEs에서 모든 이전 확산 모델보다 고품질 샘플을 생성하며, (3) 이전 보간 확산 방법인 BD3-LM과 달리 낮은 NFEs에서 모드 붕괴를 겪지 않는다는 것을 보여준다 (그림 3).

실험 설정 우리는 Eso-LM (B)를 α0train∈{0.125,0.25,1}로 학습하고, NFEs를 제어하기 위해 다양한 (α0eval,T)∈{0.0625,0.25,0.5,1}×{16,128,1024}로 샘플을 생성한다 (NFEs = |S|). MDLM과 BD3-LM은 Sahoo et al.

!

!

1 미만의 값을 초래하기 때문이다. 우리는 샘플 품질을 위해 생성 혼란도(GPT-2 Large를 통해)를 측정하고, 다양성을 위해 평균 엔트로피

생성 PPL 대 NFEs의 파레토 프론티어 그림 3에서 볼 수 있듯이, 높은 NFEs(≈ 1024)에서 Eso-LM (B)는 MDLM 및 BD3-LM을 능가하며 AR 모델 품질에 거의 필적한다. NFEs가 감소함에 따라, BD3-LM은 교사 강요 학습으로 인해 성능이 크게 저하된다: 학습 중에는 미래 블록이 깨끗한 이전 블록에 의해 조건화되지만, 평가 중에는 이전 블록 예측의 오류가 샘플 품질의 심각한 저하로 이어진다. Eso-LM은 이 문제를 피한다. 예상대로, 낮은 NFE 영역에서 더 높은 α0train (학습 중 "더 많은 확산")은 품질을 향상시킨다: α0train=1는 α0train={0.125,0.25,1} 중에서 가장 좋은 성능을 보이

며, MDLM과도 경쟁력을 유지한다. 전반적으로, Eso-LM (B)는 낮은 NFEs에서는 MDLM과, 높은 NFEs에서는 AR 모델과 경쟁력이 있다—BD3-LM과는 다르다.

생성 PPL 대 속도의 파레토 프론티어 샘플링 효율성을 비교하기 위해, 우리는 각 방법이 단일 샘플을 생성하는 데 걸리는 샘플링 시간의 중앙값(초 단위, 5회 실험 평균)을 기록하고, 그림 4에 생성 PPL 대 샘플링 시간을 플롯한다. 무한 실험의 극한에서, 중앙값 샘플링 시간은 사용된 방법과 샘플링 하이퍼파라미터에 의해 조절되는 NFEs의 증가 함수이다. 각 방법에 대해, 우리는 모든 구성에 걸쳐 파레토 프론티어를 구축한다: Eso-LM (B) (α0train, α0eval, T에 대해), BD3-LM (L′, T에 대해), MDLM (T에 대해). 우리는 Eso-LM (B)가 속도-품질 파레토 프론티어에서 새로운 최첨단 성능을 확립한다는 것을 발견했다. 샘플링 시간 예산이 낮을 때는 α0train=1로 학습된 모델이 가장 좋으며; 예산이 높을 때는 α0train=0.125 모델이 AR에 가까운 품질을 생성한다. 중간 값(예: α0train={0.25,0.5})은 속도와 품질 사이에서 최상의 절충안을 제공한다.

6 관련 연구, 논의 및 결론

AR 모델은 토큰을 왼쪽에서 오른쪽으로 생성하며 품질 면에서는 최신 기술을 유지하지만, 느리고 순차적인 추론과 제한적인 제어 가능성으로 어려움을 겪습니다. 대조적으로, Eso-LM은 순차적 단계에서 AR과 유사한 생성 방식을, 초기 확산 단계에서는 임의 순서의 병렬 생성 방식을 결합합니다. 확산 과정에서 Eso-LM (B)는 이전에는 AR 모델에만 있었던 KV 캐싱

마스크 확산 MDM [26, 30]은 단계당 여러 토큰을 생성하지만, 비용이 많이 드는 양방향 어텐션에 의존합니다. Eso-LM은 두 가지 방식으로 효율성을 개선합니다. Eso-LM (A)는 어텐션을 깨끗한(clean) 토큰과 디노이즈될(scheduled-to-denoise) 토큰으로 제한하며, Eso-LM (B)는 KV 캐싱을 위해 AO-ARM

블록 확산 BD3-LM

훈련 중 교사 강제(teacher forcing) 사용으로 인해 후속 블록의 샘플 품질에 심각한 영향을 미칩니다. Eso-LM은 이러한 문제로 어려움을 겪지 않습니다.

결론

우리는 자동 회귀(AR) 모델과 마스크 확산 모델(MDM)을 완벽하게 융합하는 새로운 언어 모델링 패러다임을 도입하여, 샘플 품질과 생성 속도 모두에서 두 모델 간의 효과적인 보간을 가능하게 합니다. 우리의 방법은 MDM에서 병렬 생성을 유지하면서 KV 캐싱을 가능하게 한 최초의 방법으로, 추론을 크게 가속화합니다. 이 방법은 블록 확산 방법을 속도와 정확도 면에서 능가하며, 언어 모델링 벤치마크에서 새로운 최첨단 기술을 확립합니다. 언어 모델링 연구를 수행하는 만큼, 이 연구 분야에 내재된 위험과 기회를 함께 고려합니다.

참고문헌

-

목차

1 서론 1 2 배경 2 2.1 자동 회귀 모델 3 2.2 마스크 확산 모델 3 2.3 블록 디노이징 확산 이산 언어 모델 3 3 에소테릭 언어 모델 4 3.1 자동 회귀 모델과 마스크 확산의 융합 4 3.2 샘플링 5 4 공유 디노이징 트랜스포머를 위한 어텐션 메커

니즘 설계 5 4.1 Eso-LM (A) 6 4.2 Eso-LM (B) 8 5 실험 9 5.1 가능도 평가 9 5.2 생성 속도 9 5.3 생성 품질 10 6 관련 연구, 논의 및 결론 11 부록 16 부록 A 배경 16 A.1 BD3-LM 하이퍼파라미터 T′ nd num_tries 16 부록 B 에소테릭 언어 모델 17 B.1 MDM 손실 17 B.2 훈련 절차 18 B.3 순차적 단계를 위한 어텐션 바이어스 18 B.4 샘플링 스케줄 및 샘플러 18 B.5 구현 세부 사항 19 B.6 실험 세부 사항 20 부록 C 추가 실험 22 C.1 분할 비율에 대한 제거 연구 22 C.2 OWT 혼란도 22 C.3 제로샷 가능도 평가 22 C.4 OWT로 훈련된 모델에 의한 생성된 샘플의 생성 혼란도 및 엔트로피 23

부록

부록 A 배경

A.1 BD3-LM 하이퍼파라미터 T′ nd num_tries

BD3-LM

또한, 우리는 BD3-LM의 코드베이스가 샘플을 선별한다는 것을 발견했습니다. 더 구체적으로 말하면, 하나의 샘플을 생성하기 위해 코드베이스는 하나의 샘플이 어떤 품질 관리 테스트를 통과할 때까지 새로운 샘플을 계속 생성합니다 (num_tries 횟수까지). 기본적으로 num_tries는 10으로 설정되어 있으며, 10번의 시도가 모두 소진되었음에도 통과하는 샘플이 없을 경우 코드베이스는 샘플링 실패를 보고합니다. 경험적으로 우리는 T′=5000의 경우 샘플링 실패가 발생하지 않는다는 것을 발견했습니다.

BD3-LM의 병렬 생성에 대한 실제 성능을 조사하기 위해, 우리는 num_tries를 1로 설정하고 품질 관리 테스트를 비활성화한 다음 다양한 T 값에 걸쳐 BD3-LM의 샘플을 평가했습니다(그림 5). 여기 그리고 그림 5에서 T는 BD3-LM의 모든 블록에 걸친 샘플링 단계의 합계를 의미하며, 예를 들어 L′=16와 T=4096는 각 블록당 T′=4096/(1024/16)=64 샘플링 단계가 사용됨을 의미합니다. 대조적으로, BD3-LM의 코드베이스는 기본적으로 T′=5000를 사용하며, 이는 그림 5에서 T=∞에 해당합니다. MDLM의 경우, 블록이 없기 때문에 T는 일반적으로 해석될 수 있습니다.

그림 5에서 볼 수 있듯이, 더 많은 병렬 생성을 가능하게 하기 위해 T가 감소할수록 BD3-LM의 샘플 품질과 샘플 다양성 모두 MDLM보다 훨씬 나빠집니다. 이러한 성능 저하는 섹션 6에서 논의된 모드 붕괴(mode collapse) 문제 때문입니다. 또한, num_tries를 늘리면 BD3-LM의 샘플 엔트로피(표 4의 두 번째 행)를 어느 정도 향상시키고 모드 붕괴를 완화할 수 있지만, AR 및 MDLM에는 개선 효과가 거의 없거나 전혀 없습니다.

!

표 4: 핵 샘플링 p=0.9을 사용한 BD3-LM L′=16, MDLM, AR의 생성 PPL (↓) 및 엔트로피 (↑) (괄호 안). 우리는

부록 B 난해한 언어 모델

B.1 MDM 손실

NLL은 다음과 같이 주어집니다:

$$-\log p_\theta(x) \le -E_{z_0 \sim q_0(\cdot|x)} \log p_\theta^{AR}(x|z_0) + D_{KL}(q_0(z_0|x) \,\|\, p_\theta^{MDM}(z_0)) = -E_{z_0 \sim q_0(\cdot|x)} [\Sigma_{\ell \in M(z_0)} \log p_\theta^{AR}(x_\ell | z_0, x_{<\ell})] + D_{KL}(q_0(z_0|x) \,\|\, p_\theta^{MDM}(z_0)). \tag{39}$$

$\mathbf{z}_0$는 인덱스 ℓ를 초과하는 인덱스에 깨끗한 토큰을 포함할 수 있습니다. 섹션 3.1에서 논의된 바와 같이, AR 로그-우도는 다음과 같이 주어집니다:

Σℓ∈M(z0)logpθAR(xℓ|z0,x<ℓ)=Σℓ∈M(z0)log(xθℓ(z0⊙x<ℓ),xℓ),₩tag40

여기서 우리는 마스킹된 인덱스 $\mathscr{M}(\mathbf{z}0)$에서만 손실을 계산합니다.

(39)의 KL 항을 계산하기 위해, 우리는 $\mathbf{z}0$에 대한 마스킹된 확산 프로세스를 정의합니다. 이 확산 프로세스의 경우, 시간 t∈[0,1]에서의 순방향 주변 $\mathbf{z}t\ell$~qt(·|$\mathbf{x}\ell$)은 (1)과 동일하지만, 축소된 범위 (αt)t∈[0,1]∈[0,α0]를 가진 노이즈 스케줄을 사용하며, 이는 αt=0=α0를 가지는 t에서 엄격하게 감소하는 함수로, $\mathbf{z}$t=0=$\mathbf{z}$0 및 αt=1=0는 $\mathbf{z}$t=1=$\mathbf{m}$1:L를 만족합니다. T 확산 단계를 통해, 다음을 얻습니다:

DKL(q0(z0|x) ‖ pθMDM(z0))=Ez0[logq0(z0|x)pθMDM(z0)]=Ez0[logEz1T:1[q(z0:1|x)pθMDM(z0:1)]]≤Ez0:1[logq(z0:1|x)pθMDM(z0:1)]=Ez0:1[Σt∈{1T,2T,⋯,1}logq(zt−1T|zt,x)pθMDM(zt−1T|zt)]=ΣtEzt[DKL(q(zt−1T|zt,x) ‖ pθMDM(zt−1T

Sahoo 등

=Et~U[0,1],zt~qt[αt′1−αtΣℓ∈M(zt)log(xθℓ(zt),xℓ)].₩tag41

마지막으로, (40)과 (41)을 결합하여 원하는 결과를 얻습니다:

LNELBO(x;θ)=Ez0~q0[−Σℓ∈M(z0)log(xθℓ(z0⊙x<ℓ),xℓ)⏟A R l o s s]
+∫t=0t=1αt′1−αtEzt~qt[Σℓ∈M(zt)log(xθℓ(zt),xℓ)]dt⏟M D M l o s s.₩tag42

B.2 훈련 절차

알고리즘 1은 완전한 훈련 절차를 요약합니다.

알고리즘 1 Eso-LM 훈련 입력: 데이터셋 D, 배치 크기 bs, 순방향 노이즈 프로세스 qt(·|$\mathbf{x}$), 모델 $\mathbf{x}$θ, 학습률 η 수렴할 때까지 반복 $\mathbf{x}$1,$\mathbf{x}$2,⋯,$\mathbf{x}$bs~D i←1부터 bs/2까지 반복 ▷ 만약 α0=1이면, 1부터 bs까지 반복. $\mathbf{z}$0~q0(·|$\mathbf{x}$) 제약 조건이 있는 σ~$\mathcal{P}$L $\mathbf{x}$θ (섹션 4)에서 어텐션 편향 A을 구성하는 데 사용됨 (7)의 순차 손실 추정기 $\mathscr{L}$←−Σℓ∈$\mathscr{M}$($\mathbf{z}$0)log($\mathbf{x}$θℓ($\mathbf{z}$0,$\mathbf{x}$<ℓ),$\mathbf{x}$iℓ) 반복 종료 i←bs/2+1부터 bs까지 반복 ▷ 만약 α0=1이면, 이 반복을 건너뜀. t~$\mathcal{U}$[0,1] 샘플링 $\mathbf{z}$t~qt(·|$\mathbf{x}$) 제약 조건이 있는 σ~$\mathcal{P}$L $\mathbf{x}$θ (섹션 4)에서 어텐션 편향 A을 구성하는 데 사용됨 (7)의 MDM 손실 추정기 $\mathscr{L}$←αi′1−αiΣℓ∈$\mathscr{M}$($\mathbf{z}$t)log($\mathbf{x}$θℓ($\mathbf{z}$t),$\mathbf{x}$iℓ) 반복 종료 θ←θ−η∇θΣi=1bs$\mathscr{L}$ 반복 종료

B.3 순차 단계의 어텐션 편향

그림 6은 순차 단계 훈련을 위한 어텐션 편향 비교를 보여줍니다.

!

B.4 샘플링 스케줄 및 샘플러

Eso-LM은 두 가지 샘플링 단계를 수행합니다: 확산 단계와 순차 단계. 확산 단계에서는 토큰이 무작위 순서로 그리고 잠재적으로 병렬로 디노이징됩니다. 순차 단계에서는 남은 마스크 토큰이 왼쪽에서 오른쪽으로, 한 번에 하나씩 순차적으로 디노이징됩니다.

먼저, (i) 확산 단계 동안 디노이징할 전체 토큰 수와 (ii) 확산 단계당 디노이징할 토큰 수를 결정하기 위해, 우리는

T 단계. 각 단계에서 우리는 시간 t에 디노이징할 토큰의 수를 다음과 같이 계산합니다:

nt=Binom(n=ntremaining,p=αs−αt1−αt), (43)

여기서 s=t−dt이고 ntremaining=L−Σt′>tnt′입니다. T가 클 때, 일부 nt는 0일 수 있습니다. 이 알고리즘에 의해 생성된 모든 nt는 0인 nt를 제외하고 정렬된 목록에 수집됩니다. 우리는 모든 nt의 합을 nMDM로 표기하고 nAR=L−nMDM로 정의합니다.

우리는 확산 방식으로 디노이징할 [L]에서 nMDM개의 토큰 인덱스를 선택하고, 보완적인 토큰 인덱스 하위 집합을 사용하여 순차적으로 디노이징합니다. 예를 들어, L=8이고 토큰 인덱스가 [1,2,⋯,8]라고 가정합시다. 위 알고리즘에서 nMDM=5를 얻었다고 가정합시다. 그러면 우리가 선택할 확산 인덱스는 (1,3,4,6,7)이고, 보완적인 순차 인덱스는 (2,5,8)입니다. 우리는 무작위 순서 디노이징을 위해 확산 인덱스를 예를 들어 (3,1,6,4,7)와 같이 무작위로 재배열합니다.

0이 아닌 nt 목록과 순열화된 확산 인덱스 정렬 집합이 주어졌을 때, nt에 따라 확산 인덱스를 분할하여 확산을 위한 샘플링 스케줄을 생성합니다. 0이 아닌 nt의 목록이 (2,1,2)이라고 가정합니다. 이를 사용하여 순열화된 확산 인덱스 집합 (3,1,6,4,7)를 분할하면, 확산 단계에 대한 다음 샘플링 스케줄을 얻습니다: $\mathcal{S}$MDM=((3,1),(6),(4,7)). 순차 단계에 대한 디노이징 스케줄은 단순히 $\mathcal{S}$AR=((2),(5),(8))입니다. 통합 샘플링 스케줄 $\mathcal{S}$은 $\mathcal{S}$MDM과 $\mathcal{S}$AR의 연결입니다. 이 예시에서는 S1=(3,1),S2=(6),S3=(4,7),S4=(2),S5=(5) 및 S6=(8)인 $\mathcal{S}$=(S1,S2,S3,S4,S5,S6)입니다. 이는 6개의 NFE에 해당합니다. 마지막으로, $\mathcal{S}$은 나머지 샘플링 절차를 처리하는 알고리즘 2로 전달됩니다. 섹션 4.1.2 및 4.2.2에서 논의된 디노이징 순서 σ로 다시 돌아가면, 이 예시에서는 σ=(3,1,6,4,7,2,5,8)입니다.

입력: 시퀀스 길이 L, 통합 샘플링 스케줄 $\mathcal{S}$ $\mathbf{z}$=[MASK_INDEX,⋯,MASK_INDEX] C={} ▷ 클린 토큰의 인덱스 i←1부터 |$\mathcal{S}$|까지 반복 ▷ |C|≥nMDM일 때 순차적으로 자동 실행 로짓 ←$\mathbf{x}_\theta(\mathbf{z}[C \cup Si])$ ▷ 비고 참조 로짓 ← Si에 해당하는 로짓 선택 $\mathbf{z}$[Si]←categorical_sample(logits, dim=-1) ▷ 로짓의 형태는 (|Si|,|$\mathcal{V}$|) C←C∪Si 반복 종료 반환: $\mathbf{z}$ 비고. $\mathbf{z}$[C∪Si]은 디노이징 모델 $\mathbf{x}_\theta$에 공급되는 $\mathbf{z}$ 내 토큰들의 부분 집합을 나타냅니다. 토큰 $\mathbf{z}_\ell \in \mathbf{z}$[C∪Si]의 위치 임베딩은 원본 시퀀스 $\mathbf{z}$에서의 그것과 동일하게 보장됩니다. Eso-LM (A) 및 Eso-LM (B)에 대한 샘플링 어텐션 편향 A 계산에 대해서는 각각 섹션 4.1.2 및 섹션 4.2.2를 참조하십시오. Eso-LM (B)의 경우, 인과적 어텐션(causal attention)을 사용하기 때문에 $\mathbf{x}_\theta$는 클린 토큰이 처음 처리될 때 KV-값을 캐시할 수 있습니다.

알고리즘 2 Eso-LM 샘플링

B.5 구현 세부 사항

B.5.1 훈련

확산 단계

Eso-LM (A)와 Eso-LM (B) 모두에서, 우리는 입력 $\mathbf{z}_t$뿐만 아니라 A의 행과 열을 σ로 정렬합니다. 이는 효율적으로 구현될 수 있는 고전적인 어텐션 편향(그림 7)을 만듭니다. 또한 토큰이 원래의 위치 임베딩을 유지하도록 $\mathbf{z}_t$의 위치 임베딩을 σ로 정렬합니다. 손실을 계산할 때, 우리는 추가적으로 목표 $\mathbf{x}$를 σ으로 정렬합니다.

순차 단계

Eso-LM (A)와 Eso-LM (B) 모두에서, 우리는 $\mathbf{z}_0$, $\mathbf{x}$ (입력 $\mathbf{z}_0 \oplus \mathbf{x}$로 연결하기 전), 그리고 A에 있는 네 개의 L×L 블록 각각의 행과 열을 σ로 정렬합니다. 또한 $\mathbf{z}_0$의 클린 토큰에 대해 특정 추가 어텐션 연결을 추가합니다. 이는 다른 토큰이 $\mathbf{z}_0$의 클린 토큰에 어텐션을 주지 않기 때문에 트랜스포머 출력에 영향을 미치지 않습니다. 이 두 가지 변환은 효율적으로 구현될 수 있는 희소하고 패턴화된 블록 구조(그림 8)를 가진 어텐션 편향을 만듭니다. 또한 토큰이 원래의 위치 임베딩을 유지하도록 $\mathbf{z}_0$과 $\mathbf{x}$의 위치 임베딩을 σ으로 정렬합니다. 손실을 계산할 때, 우리는 추가적으로 목표 $\mathbf{x}$을 σ로 정렬합니다.

!

!

B.5.2 샘플링

그림 9는 Eso-LM (A)의 샘플링 절차가 예시 시퀀스를 생성하는 방법을 보여줍니다. 그림 10은 Eso-LM (B)의 샘플링 절차가 예시 시퀀스를 생성하는 방법을 보여줍니다.

B.6 실험 세부 사항

B.6.1 저 불일치 샘플러

훈련 중 분산을 줄이기 위해 Kingma 등

B.6.2 우도 평가

우도는 t에 대한 단일 몬테카를로 추정치를 사용하여 평가합니다. 추정치의 분산을 줄이기 위해 저 불일치 샘플러

!

!

B.6.3 언어 모델링

우리는 [15, 26]에 따라 One Billion Words 데이터셋을 디토큰화했으며, 해당 코드는 여기에서 찾을 수 있습니다. Austin 등

우리는 GPT2 토크나이저로 OpenWebText를 토큰화합니다. 우리는 이를 연결하고 래핑하여 길이 1,024로 만듭니다. 래핑할 때, 연결된 시퀀스 사이에 eos 토큰을 추가합니다. One Billion Words와 달리, OpenWebText의 최종 전처리된 시퀀스는 첫 번째 및 마지막 토큰으로 특수 토큰을 가지지 않습니다. OpenWebText는 검증 분할이 없으므로, 마지막 10만 개 문서를 검증용으로 남겨둡니다.

Eso-LM은 자기회귀 기준 모델인 SEDD, MDLM, UDLM, DUO와 동일한 파라미터화를 공유합니다. 이는 Lou 등

우리는 로그-선형 노이즈 스케줄 $\alpha t = \alpha 0(1-t)$을 사용합니다. 우리는 배치 크기 512의 AdamW 옵티마이저를 사용하며, 2,500 스텝 동안 학습률 0에서 3e-4까지 일정한 학습률 웜업을 적용합니다. One Billion Words에 대해서는 100만 스텝 동안, OpenWebText에 대해서는 25만 스텝 동안 일정한 학습률을 사용합니다. 드롭아웃 비율은 0.1을 사용합니다. H200 GPU에서 모델을 훈련합니다. One Billion Words에서 100만 스텝 동안 훈련은 $\alpha 0=1$일 때 ~21시간, $\alpha 0<1$일 때 ~34시간이 소요됩니다. OpenWebText에서 25만 스텝 동안 훈련은 $\alpha 0=1$일 때 ~50시간, $\alpha 0<1$일 때 ~69시간이 소요됩니다.

부록 C 추가 실험

C.1 분할 비율에 대한 제거 연구

표 5를 참조하십시오.

표 5: 훈련 중 (7)의 MDM 손실을 평가하는 데 사용된 각 배치 내 예시의 비율과 50만 스텝 동안 훈련된 Eso-LM (A)에 대한 LM1B 테스트 혼란도(↓) 비교. 각 배치 내 나머지 예시들은 훈련 중 (7)의 AR 손실을 평가하는 데 사용됩니다.

C.2 OWT 혼란도

표 6을 참조하십시오.

표 6: 25만 스텝 및 100만 스텝 동안 훈련된 모델에 대한 OWT 테스트 혼란도(PPL; ↓). 우리는 확산 모델과 보간법에 대한 경계를 보고합니다. ǂ

C.3 제로샷 우도 평가

또한 OWT에서 훈련된 모델을 사용하여 보이지 않는 데이터셋을 얼마나 잘 모델링하는지 평가함으로써 모델의 일반화 능력을 탐구합니다 (표 7). 우리는 Eso-LM의 혼란도를 SEDD

표 7: OWT에서 25만 스텝 동안 훈련된 모델의 제로샷 혼란도(↓). 우리는 확산 모델과 보간법에 대한 경계를 보고합니다. AR에 대한 수치는

C.4 OWT에서 훈련된 모델에 의해 생성된 샘플의 생성 혼란도 및 엔트로피

그림 3에서는 NFE를 변화시킴에 따라 샘플 품질이 어떻게 변하는지 연구합니다. 생성 PPL 및 엔트로피의 수치는 표 8 (Eso-LM), 표 9 (MDLM), 표 10 (BD3-LM)에서 찾을 수 있습니다.

표 8: OWT에서 25만 스텝 동안 훈련된 Eso-LM (B)에 의해 생성된 샘플의 생성 PPL (↓) 및 엔트로피 (↑).

표 9: OWT에서 100만 스텝 동안 훈련된 MDLM에 의해 생성된 샘플의 생성 PPL (↓) 및 엔트로피 (↑).

표 10: OWT에서 100만 스텝 동안 훈련된 BD3-LM에 의해 생성된 샘플의 생성 PPL (↓) 및 엔트로피 (↑).